OurLegacy

Your family. Your story. Your legacy.

COMPLETE PRODUCT PLAN

Product vision • UI/UX design • Technical architecture • Roadmap • Claude Code implementation prompt

Working app name: OurLegacy

GitHub repository: ourlegacy

Version 1: Frontend-only • No database • No backend • No login

1. Executive summary

OurLegacy is a public, general-purpose family tree builder. Anyone should be able to create, visualize, edit, preserve, and export their family history without registering.

The first version prioritizes excellent UI/UX, a reliable interactive family tree editor, local autosave, and high-quality exports. It must not be hardcoded for one family or community.

The public website will be accessible to everyone, but each user's tree will be stored in that user's browser. Online sharing, cross-device synchronization, accounts, and real-time collaboration are future features—not part of Phase 1.

Product promise: Create a beautiful family tree, preserve the stories behind the names, and keep a backup you control.

Core goals

- Create multiple family trees without signing in.
- Add parents, partners, children, siblings, and other relatives.
- Record names, dates, photos, biographies, and personal stories.
- Navigate generations in an interactive visual canvas.
- Save automatically in the browser and recover after refresh.
- Export PNG, PDF, and JSON; import a JSON backup.
- Support English and Kannada across the UI and exports.
- Work well on desktop, tablet, and mobile.

2. Brand and design direction

Name: OurLegacy
Repository: ourlegacy
Tagline: Your family. Your story. Your legacy.

The design should feel like a premium, modern SaaS product with a warm family-history identity: minimal, welcoming, elegant, trustworthy, and easy to navigate.

Design element	Recommendation
Primary color	Forest green — #315C46
Background	Warm ivory — #F8F8F5
Cards	White — #FFFFFF
Secondary accent	Sage — #E7EFE8
Highlight	Muted gold — #C8A96B
Text	Deep charcoal — #202820
Typography	Inter or Geist; restrained serif for selected headings
Icons	Lucide
Components	Consistent rounded cards, subtle borders, restrained shadows
Motion	Smooth, purposeful transitions; reduced-motion support

Avoid excessive gradients, heavy shadows, rainbow-colored relationship lines, over-animated screens, and generic admin-dashboard styling.

3. Application screens

A. Public landing page

- Navigation: Logo, Features, How it works, English/Kannada selector, and Create your family tree.
- Hero headline: “Your family. Your story. Your legacy.”
- Supporting copy: “Discover the connections that make your family unique. Build a beautiful family tree, preserve generations of memories, and keep your family story alive.”
- Primary CTA: Create your family tree. Secondary CTA: Explore a sample tree.
- Show three benefits: No account required; saved in your browser; export to PNG and PDF.
- Display a polished fictional sample tree with several generations, person cards, relationship connectors, and photos or initials.
- Add features, how-it-works, privacy explanation, final call to action, and footer.

B. Dashboard

- Sidebar: OurLegacy logo, My Trees, Create New Tree, Import Tree, Settings, Help.
- Main area: welcome message, create-tree card, recently opened trees, and empty state.
- Each tree card shows name, number of people, last-modified date, preview, and Open/Rename/Duplicate/Delete actions.
- Keep the first-time experience simple: users should be able to start a tree immediately.

C. Family tree editor

The editor is the most important screen. On desktop, use a top toolbar, left action sidebar, central canvas, and right-side person details panel. The canvas should take the largest share of the screen.

Area	Contents
Top toolbar	Tree title, local save status, undo/redo, search, language selector, export, settings.
Left sidebar	Add parent/partner/child/sibling/relative; layout controls; zoom; fit to screen; import/export JSON.
Central canvas	Custom person cards, relationship connectors, pan/zoom, selection, centering, fit-to-screen, branch navigation.
Right details panel	Profile, dates, photo, biography, parents, partners, children, siblings, edit/add/remove actions.

Person cards should show a photo or initials, name, and birth/death years when available. Keep cards readable and avoid using gender-based colors as the main way to explain relationships.

D. Add/edit person

- Fields: first name, middle name (optional), last name, profile photo (optional), birth/death dates (optional), gender (optional), biography and notes (optional).
- Use a short, accessible modal or side panel rather than a long wizard.
- After adding: validate, create the person, create the relationship, update the canvas, select the new person, and persist.
- Never lose existing data when editing.

E. Export and settings

- Export PNG, PDF, and JSON. Support full-tree export and selected branch where feasible.
- PDF must have an intentional page-sizing strategy; large trees may require multi-page output.
- Settings include language, default layout, local data management, clear-all confirmation, privacy explanation, and app version.
- Exported files must not be limited to the currently visible viewport when full-tree export is selected.

4. Signature features

Family stories

Allow users to add a biography or personal story to each profile: where someone grew up, their occupation, important life events, or meaningful family memories. Store stories locally in Phase 1.

Generational navigation

Provide shortcuts for grandparents, parents, current generation, children, and grandchildren. Help users move through large trees without losing context.

Privacy-first experience

Show a clear notice: “Your family tree is saved in this browser. Export a backup to keep a copy safe.” Do not imply cloud storage, automatic sharing, or synchronization.

English and Kannada

Translate UI labels, menus, dialogs, errors, empty states, and accessibility labels. Test Kannada text in cards, profiles, PNG exports, and PDFs. Use locale-aware date formatting and do not assume all families follow Western naming conventions.

5. Technical architecture

Layer	Recommendation
Frontend	React + TypeScript + Vite
Styling/UI	Tailwind CSS + shadcn/ui
Icons	Lucide
Tree visualization	React Flow (@xyflow/react) or a suitable genealogy-focused renderer
State	Zustand
Local persistence	IndexedDB via Dexie
Forms and validation	React Hook Form + Zod
PNG/PDF export	Suitable SVG/DOM-to-image and PDF libraries
Unit/component tests	Vitest + React Testing Library
End-to-end tests	Playwright
Hosting	Netlify or Vercel static hosting

Explicitly out of scope: database, backend API, authentication, cloud storage, subscriptions, and real-time collaboration.

6. Family data model

Family tree

- Unique tree ID
- Tree name
- Created/updated timestamps
- People collection
- Relationships collection
- Optional layout metadata
- Schema version

Person

- Stable unique ID
- First, middle, and last name fields
- Optional birth/death dates
- Optional gender
- Optional photo reference
- Optional biography and notes

Relationship

- Stable relationship ID
- Relationship type
- Source person ID
- Target person ID

Store people and relationships separately. Support parent-child relationships, multiple parents where applicable, partners, multiple partners, siblings derived from shared parents, unknown parents, and optional adoptive relationships. Prevent self-relationships, unintended duplicates, missing references, and invalid parent-child cycles. Keep family data independent of the rendering library.

7. Local persistence and recovery

- Use IndexedDB through Dexie.
- Autosave changes with suitable debouncing.
- Show accurate states: Saving…, Saved locally, and Save failed.
- Load existing trees on startup and preserve data across refreshes.
- Support multiple trees and JSON backup/restore.
- Validate imported JSON before accepting it and confirm before overwriting existing trees.
- Handle storage failures gracefully and never silently discard data.
- Warn that browser-data deletion or device loss can cause data loss without an exported backup.
- Do not transmit family information to external services or show fake cloud-sync indicators.

8. Undo, redo, responsive design, accessibility

Undo/redo

Support undo and redo for adding/editing/deleting people, adding/removing relationships, and applicable tree settings. Avoid recording every pointer movement as a separate history entry.

Responsive behavior

- Desktop: full editor layout with persistent toolbar and side panels.
- Tablet: collapsible sidebars and flexible canvas.
- Mobile: full-screen canvas, compact toolbar, touch-friendly pan/zoom, bottom action menu, and profile drawer/bottom sheet.
- Prevent horizontal page overflow and test real touch interactions.

Accessibility

- Semantic HTML, keyboard-accessible controls, visible focus states, accessible dialogs, form labels, ARIA labels where appropriate, and sufficient contrast.
- Support reduced motion.
- Provide keyboard alternatives for important canvas actions where feasible.

9. Repository structure

```
ourlegacy/
  public/
  src/
    app/
    components/ui/
    components/layout/
    features/landing/
    features/dashboard/
    features/family-tree/
    features/people/
    features/relationships/
    features/stories/
    features/export/
    features/import/
    features/settings/
    hooks/  i18n/  lib/  services/local-storage/
    stores/  types/  utils/
  docs/
  tests/
  .github/workflows/
  README.md  package.json  tsconfig.json  vite.config.ts
```

Adapt the structure to existing repository conventions. Keep relationship logic, layout logic, persistence, import/export, and UI components separate.

10. Development roadmap

Phase	Deliverables	Priority
1. Design	Design system, landing page, responsive visual foundation	Critical
2. Editor	Canvas, person cards, relationships, selection and navigation	Critical
3. Persistence	Autosave, multiple trees, refresh recovery	Critical
4. Editing	Profile forms, search, undo/redo	Critical
5. Export	PNG, PDF, JSON import/export	Critical
6. Localization	English and Kannada throughout the app	High
7. Responsive polish	Desktop, tablet, mobile, accessibility	High
8. Testing	Unit, integration, end-to-end tests	High
9. Deployment	Static hosting, documentation, production build	High
10. Future	Accounts, cloud backup, sharing, collaboration	Later

Recommended workflow: first build the landing page and editor shell with fictional sample data. Review the design before implementing all relationship logic and persistence. This catches design-direction issues early.

11. Testing and definition of done

- Test tree creation, people editing, relationship validation, deletion, undo/redo, local persistence, reload recovery, JSON import/export, localization, and export behavior.
- End-to-end journey: open landing page → create tree → add people → verify relationships → refresh → verify recovery → export → import → verify restored data.
- Run lint, TypeScript checks, unit tests, end-to-end tests where supported, and production build.
- Review desktop/mobile layouts, keyboard navigation, Kannada rendering, export fidelity, and failure states.
- Do not mark features complete unless implemented and verified.

12. Documentation and deployment

- README.md
- docs/PRODUCT_OVERVIEW.md
- docs/UI_UX_GUIDELINES.md
- docs/ARCHITECTURE.md
- docs/FAMILY_DATA_MODEL.md
- docs/LOCAL_PERSISTENCE.md
- docs/EXPORT_IMPORT.md
- docs/TESTING.md
- docs/DEPLOYMENT.md

- docs/ROADMAP.md

Prepare a static build for Netlify or Vercel. Ensure client-side routes work after refresh, no backend environment variables are required, and privacy copy accurately explains browser-local storage.

13. Future versions

Version	Potential features
Version 1	Frontend-only, IndexedDB, import/export, English/Kannada
Version 2	Optional accounts, cloud backup, cross-device access, shareable trees
Version 3	Invite relatives, collaboration permissions, change history, shared stories

Do not implement future-version features in the current release. Design the repository and data access layer so a backend can be introduced later without rewriting the entire UI.

14. Claude Code implementation prompt

Copy the following prompt into Claude Code after opening the OurLegacy repository.

- Role: Act as a senior product designer, UI/UX architect, and senior React/TypeScript engineer.
- Mission: Build OurLegacy, a public, modern family tree builder. Tagline: “Your family. Your story. Your legacy.”
- Scope: Build a complete, working frontend. No database, backend API, authentication, or cloud storage in this version. Use IndexedDB for local persistence.
- Features: Multiple trees; people and family relationships; profile photos, dates, biographies and stories; interactive canvas; search; undo/redo; autosave; PNG/PDF/JSON export; JSON import; English/Kannada; responsive UI.
- Design: Premium, warm, minimal interface using forest green, ivory, sage, white, and muted gold. Build a landing page, dashboard, editor, profile panel, person modal, export modal, and settings. The tree canvas must be the visual focus.
- Stack: React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Lucide, Zustand, Dexie/IndexedDB, React Hook Form, Zod, Vitest, React Testing Library, and Playwright. Choose compatible stable dependencies and a suitable tree-rendering library.
- Architecture: Separate UI, people management, relationship logic, tree layout, persistence, import/export, and localization. Use a typed, versioned data model. Keep family data independent of the visualization library. Prevent invalid parent-child cycles and duplicate relationships.
- Privacy: Autosave locally, show truthful save status, support backup/restore, validate imports, and explain the risks of browser-data loss. Never imply cloud synchronization or transmit family data to external services.
- Exports: Full-tree export must not be limited to the visible viewport. Preserve names, connectors, Kannada text, and deliberate PDF sizing. Handle large trees with a suitable export strategy.
- Quality: Accessible controls, keyboard support, responsive layouts, useful error states, real functionality instead of mock buttons, and tests for relationships, persistence, undo/redo, import/export, and localization.
- Workflow: Inspect the repository first. Build the design system, landing page, and editor shell; then relationships, persistence, exports, localization, tests, and docs. Run lint, type checks, tests, and production build; fix failures.
- Documentation: Add README and docs for product overview, UI guidelines, architecture, data model, local persistence, import/export, testing, deployment, roadmap, and limitations.
- Final instruction: Do not stop after a plan or static mockup. Implement and verify a cohesive working application. Prioritize a beautiful editor, correct relationships, reliable local persistence, and high-quality exports. Do not add a backend or database.

15. Final decisions

Item	Decision
Working brand	OurLegacy (verify name/domain/trademark availability before launch)
Repository	ourlegacy
Tagline	Your family. Your story. Your legacy.
Primary experience	Interactive family tree editor
Storage	IndexedDB in the current browser
Authentication/database	Not in Phase 1
Languages	English and Kannada
Export	PNG, PDF, JSON
Hosting	Netlify or Vercel
Future direction	Optional accounts, cloud backup, and collaboration

Recommendation: Make the family tree editor genuinely beautiful and easy to use. Build the visual experience first, then implement and verify relationship logic, local persistence, exports, and responsive polish.